

**DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING**

Practical File

Web Technology (R1UC626C)

VI Semester

B. Tech CSE



Galgotias University, Greater Noida

Uttar Pradesh

India

Aviral Kumar Gautam

23SCSE1140010

Scetion - 54

LIST OF EXERCISE:

1. Write a simple HTML code to display data / content on a web page.
2. Write an HTML code to display your CV on a web page.
3. Write an HTML code to implement the concept of frames with 2 frames: one for hyperlinks and another for opening the content to that link.
4. Implement CSS using all the ways of HTML
5. Design HTML form for keeping student record and validate it using Java script.
6. Complete Web page with HTML and CSS using bootstrap
7. Write a program to implement AJAX.
8. creating a simple calculator using HTML, CSS, and JavaScript.
9. Write a program to implement jQuery.
10. Write a JSP code to generate dynamic webpage using server response.
11. Write a program using SERVLET and HTML to create a form and display the details entered by the user.
12. Write a program in SERVLET/ JSP to display content on a web page.
13. Create a SERVLET/ JSP/ JSP page for the login system using session.
14. Write a SERVLET/ JSP / JSP program for sending e-mail.

Ex.1 Write a simple HTML code to display data/content on a web page.

Aim:

To write a simple HTML code to display data/content on a web page.

HTML Code:

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Welcome to My Web Page!</h1>
  <p>This is a simple HTML page to display content.</p>
  <ul>
    <li>HTML is easy to learn.</li>
    <li>It structures web content.</li>
    <li>It is the backbone of the web.</li>
  </ul>
  <p>Thank you for visiting!</p>
</body>
</html>
```

output

When you open this file in a browser, you will see:

- A heading saying "Welcome to My Web Page!"
- A paragraph and a bulleted list below it.
- A closing paragraph saying "Thank you for visiting!"

Welcome to My Web Page!

This is a simple HTML page to display content.

- HTML is easy to learn.
- It structures web content.
- It is the backbone of the web.

Thank you for visiting!

Ex.2 Write an HTML code to display your CV on a web page.

Aim:

To write a HTML Code to Display CV on a Web Page.

HTML code :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My CV</title>
  <style>
    body {
      font-family: Arial, sans-serif; line-height: 1.6;
      margin: 20px;
    }

    h1, h2 { color:
      #333;
    }

    .section {
      margin-bottom: 20px;
    }

    .section h2 {
      border-bottom: 2px solid #333; padding-bottom: 5px;
    }
  </style>
</head>
<body>
  <h1>John Doe</h1>
  <p>Email: john.doe@example.com | Phone: +123 456 7890 | LinkedIn: <a
href="https://linkedin.com/in/johndoe">linkedin.com/in/johndoe</a></p>

  <div class="section">
    <h2>Summary</h2>
    <p>Experienced web developer with 5+ years of expertise in HTML, CSS, JavaScript, and
```

```
React. Passionate about building user-friendly and responsive web applications.</p>
</div>

<div class="section">
  <h2>Education</h2>
  <p><strong>Bachelor of Science in Computer Science</strong><br> University of
  Example, 2015 - 2019</p>
</div>

<div class="section">
  <h2>Experience</h2>
  <p><strong>Web Developer</strong><br> XYZ Company, 2019 - Present<br>
  - Developed and maintained company websites.<br>
  - Collaborated with designers to implement UI/UX designs.</p>
</div>

<div class="section">
  <h2>Skills</h2>
  <ul>
    <li>HTML, CSS, JavaScript</li>
    <li>React, Node.js</li>
    <li>Responsive Web Design</li>
    <li>Version Control (Git)</li>
  </ul>
</div>
</body>
</html>
```

Output:

John Doe Email: john.doe@example.com Phone: +123 456 7890 LinkedIn: linkedin.com/in/johndoe
Summary Experienced web developer with 5+ years of expertise in HTML, CSS, JavaScript, and React. Passionate about building user-friendly and responsive web applications.
Education Bachelor of Science in Computer Science University of Example, 2015 - 2019
Experience Web Developer XYZ Company, 2019 - Present - Developed and maintained company websites. - Collaborated with designers to implement UI/UX designs.
Skills - HTML, CSS, JavaScript - React, Node.js - Responsive Web Design - Version Control (Git)

Ex.3 Write an HTML code to implement the concept of frames with 2 frames: one for hyperlinks and another for opening the content to that link.

Aim:

To write HTML code to implement 2 frames: one for hyperlinks and another for opening the content to that link.

HTML CODE:

#frames.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Frames Example with iframe</title>
  <style>
    body {
      margin: 0;
      padding: 0; display: flex; height: 100vh;
    }

    .navigation { width: 20%; height: 100vh;
      overflow: auto;
      border-right: 2px solid #333;
    }

    .content { width: 80%; height: 100vh;
      overflow: auto;
    }

    iframe {
      width: 100%;
      height: 100%; border: none;
    }
  </style>
</head>
<body>
  <!-- Navigation Section -->
```

```
<div class="navigation">
  <iframe src="links.html" name="links"></iframe>
</div>

<!-- Content Section -->
<div class="content">
  <iframe src="content.html" name="content"></iframe>
</div>
</body>
</html>
```

`links.html` (File for Hyperlinks):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Links</title>
</head>
<body>
  <h3>Navigation</h3>
  <ul>
    <li><a href="page1.html" target="content">Page 1</a></li>
    <li><a href="page2.html" target="content">Page 2</a></li>
    <li><a href="page3.html" target="content">Page 3</a></li>
  </ul>
</body>
</html>
```

`content.html` (Default Content):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Content</title>
</head>
<body>
  <h1>Welcome!</h1>
  <p>Select a link from the left to view content.</p>
</body>
</html>
```

Example Pages (`page1.html`, `page2.html`, `page3.html`):

```
<!-- page1.html -->
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Page 1</title>
</head>
<body>
  <h1>Page 1</h1>
  <p>This is the content of Page 1.</p>
</body>
</html>
```

output:

`links.html` (File for Hyperlinks):

Navigation

- [Page 1](#)
- [Page 2](#)
- [Page 3](#)

Page 1

This is the content of Page 1.

Ex. 4 Implement CSS using all the ways of HTML

Aim:

To implement CSS using all the ways of HTML

HTML Code combining Inline CSS, Internal CSS, and External CSS (cssexample.css):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Implementation Example</title>
  <!-- Internal CSS -->
  <style> h1
    { color: blue;
      font-size: 24px;
    }

    p {
      color: green; font-size: 16px;
    }

  </style>
  <!-- External CSS -->
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <!-- Inline CSS -->
  <h1 style="color: red; font-size: 28px;">Welcome to My Website</h1>
  <p style="color: purple; font-size: 18px;">This is an example of inline CSS.</p>

  <!-- Internal CSS Example -->
  <h1>This is styled using internal CSS.</h1>
  <p>This paragraph is also styled using internal CSS.</p>

  <!-- External CSS Example -->
  <h2>This is styled using external CSS.</h2>
  <p class="external">This paragraph is styled using external CSS.</p>
```

```
</body>  
</html>
```

External CSS File (`styles.css`):

```
h2 {  
  color: orange; font-size: 22px;  
}  
  
.external { color: brown;  
  font-size: 14px;  
}
```

output:

Welcome to My Website

This is an example of inline CSS.

This is styled using internal CSS.

This paragraph is also styled using internal CSS.

This is styled using external CSS.

This paragraph is styled using external CSS.

Ex 5. Design an HTML form to keep student records and validate it using JavaScript.

Aim:

To design an HTML form to keep student records and validate it using JavaScript.

HTML CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Record Form</title>
  <style>
    body {
      background-color: #000;
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      padding-top: 10px;
    }

    .container {
      background-color: #e9e9e9;
      width: 420px;
      padding: 20px;
    }

    h2 {
      margin-bottom: 10px;
    }

    label {
      font-weight: bold;
      display: block;
      margin-top: 10px;
      margin-bottom: 5px;
    }

    input, select {
      width: 100%;
      padding: 8px;
      border: 1px solid #bdbdbd;
      box-sizing: border-box;
    }

    .btn {
      margin-top: 15px;
```

```
        background-color: #28a745;
        color: white;
        border: none;
        padding: 10px 20px;
        cursor: pointer;
        border-radius: 3px;
    }

    .btn:hover {
        background-color: #218838;
    }
</style>

<script>
    function validateForm() {
        let name = document.getElementById("name").value;
        let email = document.getElementById("email").value;
        let phone = document.getElementById("phone").value;

        if (name == "") {
            alert("Please enter your name");
            return false;
        }

        let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailPattern.test(email)) {
            alert("Enter a valid email");
            return false;
        }

        if (phone.length != 10 || isNaN(phone)) {
            alert("Enter a valid 10-digit phone number");
            return false;
        }

        alert("Form Submitted Successfully!");
        return true;
    }
</script>
</head>

<body>

<div class="container">
    <h2>Student Record Form</h2>

    <form onsubmit="return validateForm()">
```

```
<label>Full Name:</label>
<input type="text" id="name">

<label>Email:</label>
<input type="email" id="email">

<label>Date of Birth:</label>
<input type="date" id="dob">

<label>Gender:</label>
<select id="gender">
    <option>Select Gender</option>
    <option>Male</option>
    <option>Female</option>
    <option>Other</option>
</select>

<label>Phone Number:</label>
<input type="text" id="phone">

<label>Address:</label>
<input type="text" id="address">

<input type="submit" value="Submit" class="btn">

</form>
</div>

</body>
</html>
```

Output:

Student Record Form

Full Name:

Email:

Date of Birth:

dd / mm / yyyy

Gender:

Select Gender

Phone Number:

Address:

Submit

Ex.6. Creating a Complete Web Page with Bootstrap

Aim:

To create a responsive web page using HTML, CSS, and Bootstrap.

HTML CODE:

Step 1: Set Up the Basic HTML Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bootstrap Lab Exercise</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- Custom CSS -->
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <!-- Navigation Bar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
    <div class="container">
      <a class="navbar-brand" href="#">CompanyLogo</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item">
            <a class="nav-link active" href="#">Home</a>
          </li>
```

```
<li class="nav-item">
  <a class="nav-link" href="#">Features</a>
</li>
<li class="nav-item">
  <a class="nav-link" href="#">Pricing</a>
</li>
<li class="nav-item">
  <a class="nav-link" href="#">Contact</a>
</li>
</ul>
</div>
</div>
</nav>
```

```
<!-- Hero Section -->
<header class="bg-light py-5">
  <div class="container text-center">
    <h1 class="display-4">Welcome to Our Website</h1>
    <p class="lead">A responsive website built with Bootstrap</p>
    <a href="#" class="btn btn-primary btn-lg">Learn More</a>
  </div>
</header>
```

```
<!-- Features Section -->
<section class="py-5">
  <div class="container">
    <div class="row text-center">
<div class="col-md-4 mb-4">
<div class="card h-100">
<div class="card-body">
      <i class="bi bi-speedometer2 fs-1 text-primary"></i>
      <h3 class="card-title">Fast Performance</h3>
<p class="card-text">Our website is optimized for speed and performance across all
devices.</p>
    </div>
  </div>
</div>
</div>
<div class="col-md-4 mb-4">
  <div class="card h-100">
    <div class="card-body">
      <i class="bi bi-phone fs-1 text-primary"></i>
      <h3 class="card-title">Mobile First</h3>
<p class="card-text">Designed with mobile-first approach to ensure great experience on any
device.</p>
    </div>
```

```
    </div>
  </div>
  <div class="col-md-4 mb-4">
    <div class="card h-100">
      <div class="card-body">
        <i class="bi bi-shield-check fs-1 text-primary"></i>
        <h3 class="card-title">Secure</h3>
        <p class="card-text">We prioritize security to protect your data and
```


privacy.</p>

</div>

</div>

</div>

</div>

</section>

</div>

<!-- Call to Action -->

<section class="bg-primary text-white py-5">

<div class="container text-center">

<h2 class="mb-4">Ready to get started?</h2>

<button class="btn btn-light btn-lg">Sign up today</button>

</div>

</section>

<!-- Footer -->

<footer class="bg-dark text-white py-4">

<div class="container">

<div class="row">

<div class="col-md-6">

<h5>About Us</h5>

<p>We are a company dedicated to creating amazing web experiences using Bootstrap.</p>

</div>

<div class="col-md-3">

<h5>Quick Links</h5>

<ul class="list-unstyled">

Home

Features

Pricing

Contact

</div>

<div class="col-md-3">

<h5>Contact</h5>

<address>

123 Main St

City, State 12345
 info@example.com

</address>

</div>

</div>

</div>

<hr>

<div class="text-center">

<p class="mb-0">© 2023 Company Name. All rights reserved.</p>

```

</div>
</footer>

<!-- Bootstrap JS Bundle with Popper -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></scrip t>
<!-- Bootstrap Icons -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.10.0/font/bootstrap-icons.css">
</body>
</html>
```

```

## Step 2: Add Custom CSS (styles.css)

```

/* Custom styles for our Bootstrap page */

/* Add some padding to the body to account for fixed navbar */ body
{ padding-top: 56px;
}

/* Custom hero section styles */
.hero {
background: linear-gradient(rgba(0, 0, 0, 0.5), rgba(0, 0, 0, 0.5)),
url('https://via.placeholder.com/1920x600');
background-size: cover;
background-position: center; color: white;
padding: 100px 0;
}

/* Card hover effect */
.card {
transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
transform: translateY(-5px);
box-shadow: 0 10px 20px rgba(0, 0, 0, 0.1);
}

/* Custom button styles */
.btn-custom {
padding: 10px 25px; border-radius: 50px;
}

```

```

/* Footer link hover effect */ footer a
{ transition: color 0.3s ease;
}

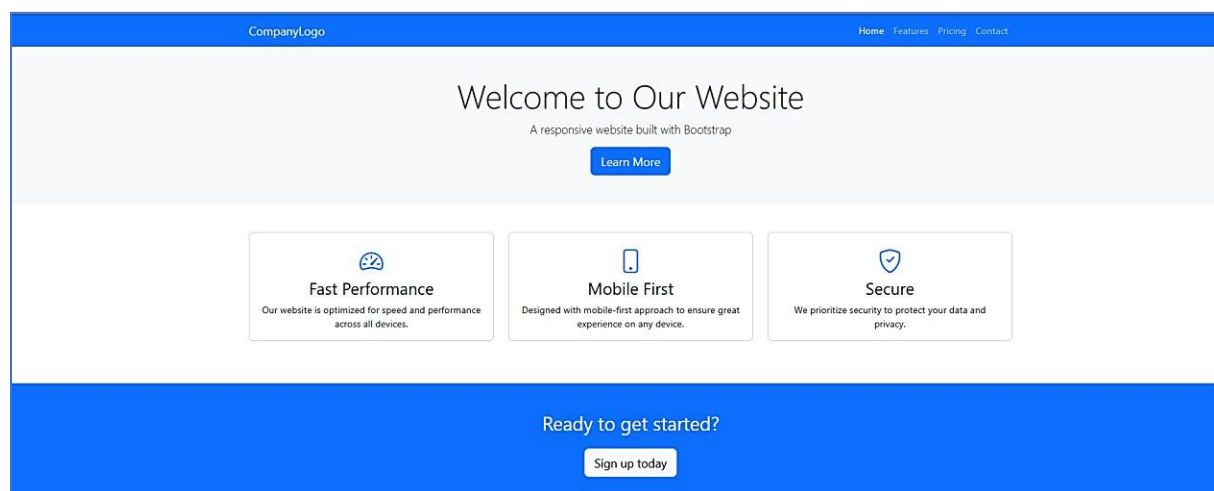
footer a:hover {
color: #0d6efd !important; text-decoration: none;
}

/* Responsive adjustments */ @media (max-width: 768px) {
.hero {
padding: 60px 0;
}

body {
padding-top: 72px;
}
}

```

## output:



# Welcome to Our Website

A responsive website built with Bootstrap

[Learn More](#)



## Fast Performance

Our website is optimized for speed and performance across all devices.



## Mobile First

Designed with mobile-first approach to ensure great experience on any device.



## Secure

## Ex. 7 Write a program to implement AJAX.

### Aim:

To implement AJAX using JavaScript's `XMLHttpRequest` and the newer `fetch()` API.

### Code:

Option 1: Using XMLHttpRequest (Traditional Approach)

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>AJAX Lab Exercise</title>
 <style>
 body {
 font-family: Arial, sans-serif;
 max-width: 800px;
 margin: 0 auto;
 padding: 20px;
 }
 #output, #fetchOutput
 { margin-top: 20px;
 padding: 15px;
 border: 1px solid #ddd;
 border-radius: 5px;
 min-height: 100px;
 }
 button {
 padding: 10px 15px;
 background-color: #4CAF50;
 color: white;
 border: none;
 border-radius: 4px;
 cursor: pointer;
 margin-right: 10px;
 }
 button:hover {
 background-color: #45a049;
 }
 </style>
</head>
<body>
 <h1>AJAX Implementation Lab</h1>

 <h2>1. Using XMLHttpRequest</h2>
```

```
<button onclick="loadDataXHR()">Load Data with XHR</button>
<div id="output">Data will appear here...</div>
```

## <h2>2. Using Fetch API</h2>

```
<button onclick="loadDataFetch()">Load Data with Fetch</button>
<div id="fetchOutput">Data will appear here...</div>
```

```
<script>
```

```
 // Using XMLHttpRequest
```

```
 function loadDataXHR() {
```

```
 const outputDiv = document.getElementById('output');
```

```
 outputDiv.innerHTML = 'Loading...';
```

```
 // 1. Create an XHR object
```

```
 const xhr = new XMLHttpRequest();
```

```
 // 2. Configure the request
```

```
 xhr.open('GET', 'https://jsonplaceholder.typicode.com/posts/1', true);
```

```
 // 3. Set up callback for when request completes
```

```
 xhr.onload = function() {
```

```
 if (this.status === 200) {
```

```
 // 4. Parse the response and display it
```

```
 const response = JSON.parse(this.responseText);
```

```
 outputDiv.innerHTML = `
```

```
 Title: ${response.title}

```

```
 Body: ${response.body}
```

```
 `;
```

```
 } else {
```

```
 outputDiv.innerHTML = 'Error loading data';
```

```
 }
```

```
 };
```

```
 // 5. Handle errors
```

```
 xhr.onerror = function() {
```

```
 outputDiv.innerHTML = 'Request failed';
```

```
 };
```

```
 // 6. Send the request
```

```
 xhr.send();
```

```
 }
```

```
 // Using Fetch API (modern approach)
```

```
 function loadDataFetch() {
```

```
 const outputDiv = document.getElementById('fetchOutput');
```

```
 outputDiv.innerHTML = 'Loading...';
```

```
 // 1. Make the fetch request
```

```
 fetch('https://jsonplaceholder.typicode.com/posts/2')
```

```

// 2. Convert response to JSON
.then(response =>
 { if (!response.ok)
 {
 throw new Error('Network response was not ok');
 }
 return response.json();
 })
// 3. Process the data
.then(data =>
 { outputDiv.innerHTML
 = `
 Title: ${data.title}

 Body: ${data.body}
 `;
 })
// 4. Handle errors
.catch(error => {
 outputDiv.innerHTML = `Error: ${error.message}`;
});
}
</script>
</body>
</html>

```

## Option 2: Using Fetch API with Async/Await (Modern Approach)

```

<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>AJAX with Async/Await</title>
 <style>
 body {
 font-family: Arial, sans-serif;
 max-width: 800px;
 margin: 0 auto;
 padding: 20px;
 }
 #userData {
 margin-top: 20px;
 padding: 15px;
 border: 1px solid #ddd;
 border-radius: 5px;
 }
 table {
 width: 100%;
 }
 </style>

```

```

 border-collapse: collapse;
 margin-top: 20px;
 }
 th, td {
 border: 1px solid #ddd;
 padding: 8px;
 text-align: left;
 }
 th {
 background-color: #f2f2f2;
 }
</style>
</head>
<body>
 <h1>User Data Loader</h1>
 <button onclick="loadUserData()">Load User Data</button>
 <div id="userData">Click the button to load user data...</div>

 <script>
 async function loadUserData() {
 const outputDiv = document.getElementById('userData');
 outputDiv.innerHTML = 'Loading user data...';

 try {
 // 1. Make the API request
 const response = await fetch('https://jsonplaceholder.typicode.com/users');

 // 2. Check if request was successful
 if (!response.ok) {
 throw new Error(`HTTP error! status: ${response.status}`);
 }

 // 3. Parse the JSON data
 const users = await response.json();

 // 4. Display the data in a table
 let html = '<h3>User List</h3><table>';
 html += '<tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th></tr>';

 users.forEach(user =>
 { html += `
 <tr>
 <td>${user.id}</td>
 <td>${user.name}</td>
 <td>${user.email}</td>
 <td>${user.phone}</td>
 </tr>
 `;
 });
 }
 }
 </script>

```



```
});

html += '</table>';
outputDiv.innerHTML = html;

} catch (error) {
 // 5. Handle any errors
 outputDiv.innerHTML = `Failed to load data: ${error.message}`;
}
}
</script>
</body>
</html>
```

**output:**

## Simple AJAX Examples

### 1. Using XMLHttpRequest

Load Data (XHR)

### 2. Using Fetch API

Load Data (Fetch)

Results will appear here...

## Ex 8. creating a simple calculator using HTML, CSS, and JavaScript.

### Aim:

To create a simple calculator using HTML, CSS, and JavaScript.

### HTML CODE:

```
<!DOCTYPE html>
<html>
<head>
 <title>Simple Calculator</title>

 <style>
 body {
 margin: 0;
 background-color: #f4f4f4;
 font-family: Arial, sans-serif;
 }

 .container {
 width: 700px;
 margin: 15px auto;
 padding: 20px;
 background-color: #ececce;
 border-radius: 10px;
 box-shadow: 0 0 10px rgba(0,0,0,0.2);
 }

 input {
 width: 100%;
 padding: 12px;
 margin-bottom: 10px;
 font-size: 18px;
 border: 1px solid #ccc;
 border-radius: 5px;
 box-sizing: border-box;
 }
```

```
button {
 width: 48%;
 padding: 12px;
 margin: 6px 0;
 font-size: 20px;
 color: white;
 background-color: #007bff;
 border: none;
 border-radius: 5px;
 cursor: pointer;
 display: block;
}

button:hover {
 background-color: #0056b3;
}

#result {
 margin-top: 15px;
 font-size: 22px;
 font-weight: bold;
}
</style>

<script>
function getValues() {
 let num1 = parseFloat(document.getElementById("num1").value);
 let num2 = parseFloat(document.getElementById("num2").value);

 if (isNaN(num1) || isNaN(num2)) {
 alert("Please enter both numbers");
 return null;
 }

 return { num1, num2 };
}
```

```
}
```

```
function sum() {
 let values = getValues();
 if (!values) return;
 document.getElementById("result").innerHTML =
 "Result: " + (values.num1 + values.num2);
}
```

```
function product() {
 let values = getValues();
 if (!values) return;
 document.getElementById("result").innerHTML =
 "Result: " + (values.num1 * values.num2);
}
```

```
function difference() {
 let values = getValues();
 if (!values) return;
 document.getElementById("result").innerHTML =
 "Result: " + (values.num1 - values.num2);
}
```

```
function quotient() {
 let values = getValues();
 if (!values) return;

 if (values.num2 == 0) {
 alert("Cannot divide by zero");
 return;
 }

 document.getElementById("result").innerHTML =
 "Result: " + (values.num1 / values.num2);
}
```

```
</script>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
 <input type="number" id="num1" placeholder="Enter first number">
```

```
 <input type="number" id="num2" placeholder="Enter second number">
```

```
 <button onclick="sum()">Sum</button>
```

```
 <button onclick="product()">Product</button>
```

```
 <button onclick="difference()">Difference</button>
```

```
 <button onclick="quotient()">Quotient</button>
```

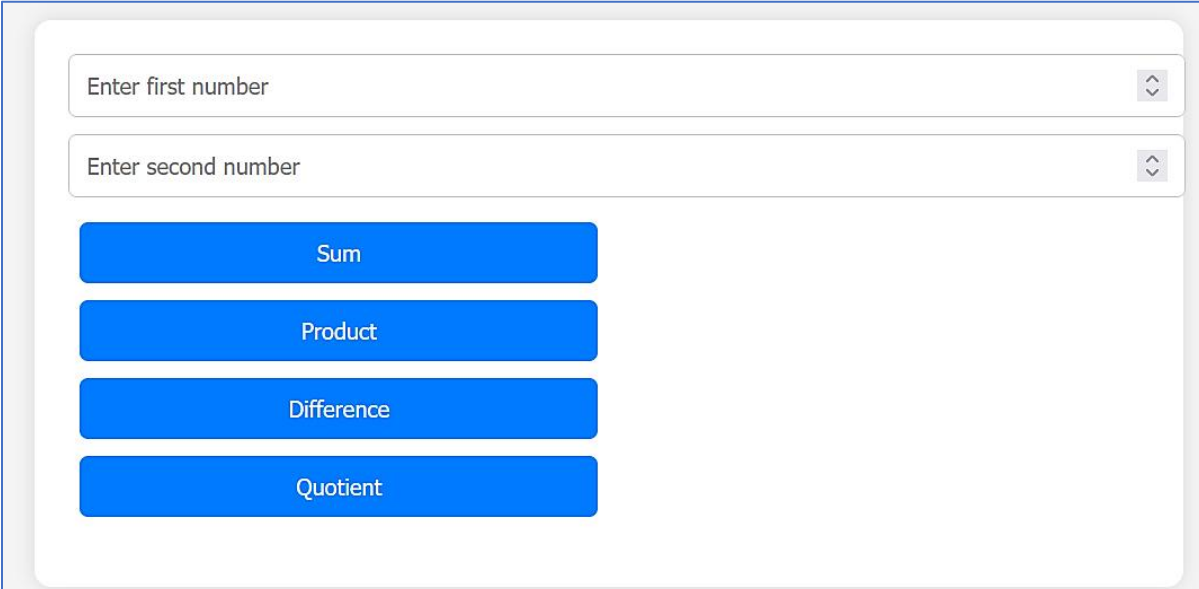
```
 <div id="result"></div>
```

```
</div>
```

```
</body>
```

```
</html>
```

## Output



The screenshot shows a web application interface with a light gray background. It features two input fields at the top, each with a placeholder text and a small up/down arrow icon on the right. Below the input fields are four blue buttons stacked vertically, each with white text. The buttons are labeled 'Sum', 'Product', 'Difference', and 'Quotient' from top to bottom. The entire interface is enclosed in a thin blue border.

Enter first number

Enter second number

Sum

Product

Difference

Quotient

## Ex.9 To implement a basic jQuery program

### Aim

To implement a basic jQuery program that manipulates HTML elements and responds to user interactions, such as button clicks, using the jQuery library.

### HTML Code: jQuery Example – Show/Hide Paragraph

```
<!DOCTYPE html>
<html>
<head>
 <title>jQuery Example</title>
 <!-- Step 1: Include jQuery Library -->
 <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

 <style>
 #message
 { color:
 green;
 font-size: 20px;
 margin-top: 20px;
 }
 </style>

 <script>
 // Step 3: Write jQuery logic after the document is ready
 $(document).ready(function () {
 $("#hideBtn").click(function () {
 $("#message").hide();
 });

 $("#showBtn").click(function () {
 $("#message").show();
 });
 });
 </script>
</head>
<body>
 <h2>jQuery Show/Hide Example</h2>
 <!-- Step 2: Create Elements -->
 <p id="message">This is a message controlled by jQuery.</p>

 <button id="hideBtn">Hide Message</button>
 <button id="showBtn">Show Message</button>
</body>
</html>
```

## Output

### jQuery Show/Hide Example

This is a message controlled by jQuery.

Hide Message

Show Message

## Ex. 10 Implement a JSP page that generates a dynamic response on the server side using user input.

### Aim:

To implement a **JSP page** that generates a **dynamic response** on the server side using user input and system date/time.

### Code – userGreeting.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="java.util.Date" %>
<!DOCTYPE html>
<html>
<head>
 <title>Dynamic Greeting Page</title>
</head>
<body>
 <h2>Dynamic JSP Page - Server Response</h2>

 <form method="post">
 Enter your name: <input type="text" name="username" required>
 <input type="submit" value="Greet Me">
 </form>

 <hr>

 <%
 String name = request.getParameter("username");
 if (name != null && !name.trim().equals("")) {
 Date currentDate = new Date();
 %>
 <h3>Hello, <%= name %>!</h3>
 <p>Welcome to our website.</p>
 <p>Current Server Date and Time: <%= currentDate %></p>
 <%
 }
 %>
 %>
 </body>
</html>
```



## Output

---

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%> <%@ page import="java.util.Date" %>
```

### Dynamic JSP Page - Server Response

Enter your name:

---

```
<% String name = request.getParameter("username"); if(name != null && !name.trim().equals("")) { Date currentDate = new Date(); %>
```

**Hello, <%= name %>!**

Welcome to our website.

Current Server Date and Time: <%= currentDate %>

```
<% } %>
```

## **Ex.11 create a form and display user-entered details using Servlet and HTML.**

### **Aim:**

To develop a web application using Servlet and HTML that collects user input from a form and displays the submitted details using a Servlet.

### **1. HTML Code – form.html**

```
<!DOCTYPE html>
<html>
<head>
 <title>User Form</title>
</head>
<body>
 <h2>User Information Form</h2>
 <form action="FormServlet" method="post">
 Name: <input type="text" name="username">

 Email: <input type="email" name="email">

 Gender:
 <input type="radio" name="gender" value="Male"> Male
 <input type="radio" name="gender" value="Female"> Female

 <input type="submit" value="Submit">
 </form>
</body>
</html>
```

### **2. Servlet Code – FormServlet.java**

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FormServlet extends HttpServlet {
 protected void doPost(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {

 // Retrieve form data
 String name = request.getParameter("username");
 String email = request.getParameter("email");
 String gender = request.getParameter("gender");

 // Set response content type
 response.setContentType("text/html");
 // Print output
 PrintWriter out = response.getWriter(); out.println("<html><body>");
```

```
 out.println("<h2>User Details Submitted:</h2>"); out.println("Name: " + name +
 "
"); out.println("Email: " + email + "
"); out.println("Gender: " + gender +
 "
"); out.println("</body></html>");
 }
}
```

### 3. Deployment Descriptor – web.xml

```
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
 <servlet>
 <servlet-name>FormServlet</servlet-name>
 <servlet-class>FormServlet</servlet-class>
 </servlet>
 <servlet-mapping>
 <servlet-name>FormServlet</servlet-name>
 <url-pattern>/FormServlet</url-pattern>
 </servlet-mapping>
</web-app>
```

#### Output:

If the user submits:



A screenshot of a web form. It contains two text input fields. The first field is labeled 'Name:' and contains the text 'John Doe'. The second field is labeled 'Email:' and contains the text 'John@example.com'. Below these fields is a button labeled 'Submit'.

The Servlet response:



A screenshot of the Servlet response. It displays the text 'Hello, John Doe' in a large, bold, black serif font. Below this, it displays 'Your email: John@example.com' in a smaller, black serif font.

## Ex. 12 Servlet/JSP Program to Display Content on a Web Page

### Aim:

To create a web application using Servlet and JSP that displays dynamic content on a web page, demonstrating the interaction between servlets and JSP pages.

### Code:

#### 1. DisplayServlet.java (Controller)

```
import java.io.IOException;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.Date;
```

```
public class DisplayServlet extends HttpServlet {
```

```
 protected void doGet(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {
```

```
 // Set dynamic content as request attributes
 request.setAttribute("title", "Welcome to Our Website");
 request.setAttribute("currentDate", new Date());
 request.setAttribute("visitorCount", 1024);
 request.setAttribute("features", new String[] {
 "Responsive Design",
 "User-Friendly Interface",
 "Secure Transactions",
 "24/7 Support"
 });
```

```
 // Forward to JSP page for display
 RequestDispatcher dispatcher = request.getRequestDispatcher("display.jsp");
 dispatcher.forward(request, response);
 }
}
```

#### 2. display.jsp (View)

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ page import="java.util.Arrays" %>
<html>
<head>
```

```

<title><%= request.getAttribute("title") %></title>
<style>
 body {
 font-family: Arial, sans-serif;
 margin: 40px;
 background-color: f5f5f5;
 }
 .container {
 background-color: white;
 padding: 20px;
 border-radius: 5px;
 box-shadow: 0 2px 5px rgba(0,0,0,0.1);
 }
 h1 {
 color: 2c3e50;
 }
 .feature-list
 { margin: 20px 0;
 padding-left: 20px;
 }
 .footer {
 margin-top: 20px;
 font-size: 0.8em;
 color: 7f8c8d;
 }
</style>
</head>
<body>
 <div class="container">
 <h1><%= request.getAttribute("title") %></h1>

 <p>Current Date and Time: <%= request.getAttribute("currentDate") %></p>
 <p>Total Visitors: <%= request.getAttribute("visitorCount") %></p>

 <h3>Our Features:</h3>
 <ul class="feature-list">
 <%
 String[] features = (String[]) request.getAttribute("features");
 for (String feature : features) {
 %>
 <%= feature %>
 <% } %>

 </div>

```

```


<div class="footer">
 Page generated by <%= this.getClass().getSimpleName() %>
</div>
</div>
</body>
</html>
```

### 3. web.xml (Deployment Descriptor)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
 http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
 version="4.0">

 <servlet>
 <servlet-name>DisplayServlet</servlet-name>
 <servlet-class>DisplayServlet</servlet-class>
 </servlet>

 <servlet-mapping>
 <servlet-name>DisplayServlet</servlet-name>
 <url-pattern>/display</url-pattern>
 </servlet-mapping>

 <welcome-file-list>
 <welcome-file>display.jsp</welcome-file>
 </welcome-file-list>
</web-app>
```

### Output:

## Welcome to Our Website

Current Date and Time: Tue Jul 01 15:00:43 IST 2025

Total Visitors: **1024**

### Our Features:

- Responsive Design
- User-Friendly Interface
- Secure Transactions
- 24/7 Support

Page generated by JSP

## Ex.13 Develop a login system using JSP and session management

### Aim

To develop a **login system using JSP and session management** that validates user credentials and allows access to a welcome page if login is successful.

### 1. login.jsp – Login Form Page

```
<!DOCTYPE html>
<html>
<head>
 <title>Login Page</title>
</head>
<body>
 <h2>Login</h2>
 <form action="loginProcess.jsp" method="post">
 Username: <input type="text" name="username" required>

 Password: <input type="password" name="password" required>

 <input type="submit" value="Login">
 </form>

 <%
 String error = request.getParameter("error");
 if ("true".equals(error)) {
 %>
 <p style="color:red;">Invalid Username or Password</p>
 <% } %>
</body>
</html>
```

### 2. loginProcess.jsp – Processing Login

```
<%@ page import="javax.servlet.http.*;javax.servlet.*" %>
<%
 String username = request.getParameter("username");
 String password = request.getParameter("password");

 // Dummy check – Replace with DB check for real-world use
 if("admin".equals(username) && "admin123".equals(password)) {
 // Create session
 HttpSession session = request.getSession();
 session.setAttribute("username", username);
 response.sendRedirect("welcome.jsp");
 } else {
 response.sendRedirect("login.jsp?error=true");
 }
}
```

%>

### 3. welcome.jsp – Welcome Page with Session Access

```
<%@ page import="javax.servlet.http.*,javax.servlet.*" %>
<%
 HttpSession session = request.getSession(false);

 if (session == null || session.getAttribute("username") == null)
 { response.sendRedirect("login.jsp");
 }
%>

<!DOCTYPE html>
<html>
<head>
 <title>Welcome</title>
</head>
<body>
 <h2>Welcome, <%= session.getAttribute("username") %>!</h2>
 Logout
</body>
</html>
```

### 4. logout.jsp – Session Destroy

```
<%@ page import="javax.servlet.http.*,javax.servlet.*" %>
<%
 HttpSession session = request.getSession(false);
 if (session != null) {
 session.invalidate();
 }
 response.sendRedirect("login.jsp");
%>
```

## Output Scenarios

### 1. Accessing login.jsp

#### Login

Username:

Password:

Login

```
<% String error =
request.getParameter("error");
if("true".equals(error)){ %>
```

Invalid Username or Password

```
<% } %>
```

### 2.



### 3. On correct login (admin / admin123)



### 4. On incorrect login



## **Ex.14 Implement sending an email using JSP/Servlet with the JavaMail API.**

### **Aim:**

To develop a JSP/Servlet-based email sending application using JavaMail API, allowing users to enter recipient details and send a message via SMTP.

### **Code**

#### **1. emailForm.jsp – Email Input Form**

```
<!DOCTYPE html>
<html>
<head>
 <title>Send Email</title>
</head>
<body>
 <h2>Send Email Form</h2>
 <form action="SendEmailServlet" method="post">
 To: <input type="email" name="to" required>

 Subject:
 <input type="text" name="subject" required>

 Message: <textarea name="message" rows="6" cols="40"
required></textarea>

 <input type="submit" value="Send Email">
 </form>
</body>
</html>
```

#### **2. SendEmailServlet.java – Servlet to Send Email**

```
import java.io.IOException;
import java.util.Properties;
import javax.mail.*;
import javax.mail.internet.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SendEmailServlet extends HttpServlet {
 protected void doPost(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {

 String to = request.getParameter("to");
 String subject = request.getParameter("subject");
 String messageText = request.getParameter("message");

 // Sender's email and password (use app password for Gmail)
 final String from = "your_email@gmail.com";
 final String password = "your_app_password";
```

```

// SMTP configuration
Properties props = new Properties();
props.put("mail.smtp.host", "smtp.gmail.com");
props.put("mail.smtp.port", "587");
props.put("mail.smtp.auth", "true");
props.put("mail.smtp.starttls.enable", "true");

// Authenticate and send
Session session = Session.getInstance(props, new Authenticator()
 { protected PasswordAuthentication getPasswordAuthentication() {
 return new PasswordAuthentication(from, password);
 }
});

try {
 Message message = new MimeMessage(session);
 message.setFrom(new InternetAddress(from));
 message.setRecipients(Message.RecipientType.TO, InternetAddress.parse(to));
 message.setSubject(subject);
 message.setText(messageText);

 Transport.send(message);

 response.setContentType("text/html");
 response.getWriter().println("<h2>Email Sent Successfully!</h2>");
} catch (MessagingException e)
 { throw new RuntimeException(e);
 }
}
}

```

### 3. web.xml – Servlet Mapping

```

<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
 <servlet>
 <servlet-name>SendEmailServlet</servlet-name>
 <servlet-class>SendEmailServlet</servlet-class>
 </servlet>
 <servlet-mapping>
 <servlet-name>SendEmailServlet</servlet-name>
 <url-pattern>/SendEmailServlet</url-pattern>
 </servlet-mapping>
</web-app>

```

## Output:

### 1. Email Form Page (emailForm.jsp)

← → ↻ http://localhost:8080/EmailApp/emailForm.jsp ☆ ⓘ ⋮

#### Send Email Form

To:

Subject:

Message:

### 2. After Entering Details

← → ↻ http://localhost:8080/EmailApp/emailForm.jsp ☆ ⓘ ⋮

#### Send Email Form

To:

Subject:

Message:

### 3. After Clicking "Send Email"

← → ↻ http://localhost:8080/EmailApp/SendEmailServlet ☆ ⓘ ⋮

**Email Sent Successfully!**